LLM EVALUATION & BENCHMARKING PLATFORM

# Software Development Plan

*(SDP)*

Document Version 1.0
September 2026

**Prepared by**
Monish D — 24BDS1033
Soaham S — 24BDS1021

Project: LLM Evaluation and Benchmarking Platform
Course: Software Engineering — Digital Assignment

# Table of Contents

**Table of Contents** ..... 2
**1. Introduction** ..... 3
**1.1 Purpose** ..... 3
**1.2 Development Approach** ..... 3
**2. Project Organization** ..... 3
**2.1 Team Roles** ..... 3
**2.2 Tools** ..... 3
**3. Phased Schedule** ..... 3
**4. Environment & Infrastructure Plan (Zero-Budget Constraint)** ..... 4
**5. Risk Management** ..... 5
**6. Quality Assurance & Testing Strategy** ..... 5
**7. Documentation Plan** ..... 5

# 1. Introduction

## 1.1 Purpose

This Software Development Plan (SDP) defines how the LLM Evaluation and Benchmarking Platform will be built: process model, phased schedule, team roles, tooling, environment, risk management, and quality assurance approach. It operationalizes the phases outlined in the source project report into a concrete, zero-budget, bottom-up execution plan.

## 1.2 Development Approach

The project will follow an iterative, bottom-up incremental methodology: core evaluation logic and data model are built and validated first (without any UI), followed by the API layer, followed by the frontend, followed by the industry-comparison and reporting layers, followed by deployment. Each increment is independently testable before the next is layered on top, which suits a two-person academic team better than a heavyweight Scrum process while still using short iteration cycles (roughly weekly) with a running backlog.

# 2. Project Organization

## 2.1 Team Roles

| Member | Primary Responsibility |
|---|---|
| Monish D (24BDS1033) | Backend, Evaluation Engine, LLM Integration Layer, Industry Comparison Engine, Deployment (Railway) |
| Soaham S (24BDS1021) | Data model & database, Benchmark Manager, Dataset curation, Reporting module, Frontend integration (Stitch-generated UI) |

Both members jointly own requirements, architecture decisions, and testing; role split above indicates primary ownership only.

## 2.2 Tools

- Version control: GitHub (private repo, feature branches, PR review)
- IDE / AI coding agent: Antigravity (backend/full-stack build) and Codex-connected Stitch (frontend generation)
- Project tracking: GitHub Projects / Issues (Kanban board)
- API testing: Postman
- Containerization (optional): Docker for local parity with deployment
- CI: GitHub Actions (lint + unit tests on push)

# 3. Phased Schedule

The schedule below follows a bottom-up order: data & evaluation core → backend API → frontend → industry comparison & reporting → hardening → deployment. Durations are indicative for an academic-timeline, two-person team.

| Phase | Deliverable | Duration |
|---|---|---|
| P0 — Requirements & Design | URS, SRS, SDP, SAD, PRD (this document set); finalized data model | 1 week |
| P1 — Core Data Layer | Database schema, ORM models, seed data for datasets & baseline models | 3–4 days |
| P2 — LLM Integration Layer | Provider adapters (OpenRouter free models, Gemini free tier, self-hosted fallback) behind a common interface; retry/rate-limit handling | 4–5 days |
| P3 — Evaluation Engine (quality) | Exact-match/F1/semantic-similarity scorers; LLM-as-a-Judge scorer for open-ended tasks | 4–5 days |
| P4 — Hallucination & Reliability Module | Claim extraction, claim verification, groundedness & consistency scoring | 5–6 days |
| P5 — Industry Comparison Engine | Baseline registry, normalization (z-score/percentile), tiering, capability profile generation, positioning-statement generator | 5–6 days |
| P6 — Efficiency & Cost Module | Latency capture, token accounting, pricing-table-based cost estimation | 2–3 days |
| P7 — Backend API | FastAPI endpoints for all of the above, run orchestration, async job queue | 4–5 days |
| P8 — Frontend (Stitch/UI UX Pro Max AgentKit) | Model/dataset selection, run monitor, results dashboard, capability radar, drill-down, export | 1–1.5 weeks |
| P9 — Integration & Testing | Frontend↔backend integration, unit + integration + a small end-to-end benchmark run | 4–5 days |
| P10 — Deployment | Railway deployment of backend + frontend + managed free Postgres (Supabase/Neon) | 2–3 days |
| P11 — Performance/Validation Run | Full benchmark across several free-tier models to validate the platform end-to-end and produce a sample report | 2–3 days |

# 4. Environment & Infrastructure Plan (Zero-Budget Constraint)

All infrastructure choices are constrained to free tiers, consistent with the project having no budget.

| Need | Free Option(s) |
|---|---|
| App hosting (backend + frontend) | Railway free/hobby tier (primary target for deployment); alternatives: Render free web service, Fly.io free allowance |
| Database | Supabase free Postgres, or Neon free Postgres; SQLite for local dev |
| LLM inference for models under test | OpenRouter free-tier models, Google AI Studio/Gemini free tier, NVIDIA NIM free API credits, Cloudflare Workers AI free allocation, Groq free tier for fast open-weight models |
| LLM-as-a-Judge model | A free-tier or low-cost model from the above providers, documented per run for reproducibility |
| Object/report storage | Supabase Storage free tier, or simply generated on-demand and streamed to the client |
| CI/CD | GitHub Actions free minutes for public/small private repos |

| Need | Free Option(s) |
|---|---|
| Frontend prototyping | Stitch (Google) with the UI UX Pro Max AgentKit, connected to Codex, free/no-cost tier |

Because free-tier providers rotate model availability and rate limits change, the LLM Integration Layer must treat the provider list as configuration, not hard-coded logic, and must gracefully degrade (skip/flag) a model whose free quota is exhausted mid-run.

# 5. Risk Management

| Risk | Impact | Mitigation |
|---|---|---|
| Free-tier API rate limits / quota exhaustion mid-benchmark | Partial/incomplete results | Async queue with retry & backoff; mark failed cells instead of aborting run (FR-3.3); cache baseline scores instead of re-computing them every run |
| LLM-as-a-Judge bias skews quality/hallucination scores | Inaccurate comparative claims | Use multiple judge models where feasible; document judge identity and rubric with every score; allow reference-based scoring to override judge scoring when a ground truth exists |
| "Industry standard" baseline becomes stale as new models ship | Misleading positioning | Version and date-stamp every baseline score; schedule periodic baseline refresh; surface the baseline's age in the UI |
| Free hosting/database limits (storage, cold starts, sleep) affect availability | Downtime, slow first request | Keep the dataset/result footprint small; use a lightweight backend framework; document expected cold-start behavior |
| Two-person team, academic deadline pressure | Schedule slip | Strict bottom-up phase order so a usable core exists even if later phases (e.g., polished UI) run late |
| Open-source repos referenced for design ideas may have incompatible licenses | Legal/academic-integrity risk if code is copied | Treat cloned reference repos as read-only inspiration in a clearly separated /reference folder; do not copy code verbatim into the shipped project; cite anything adapted |

# 6. Quality Assurance & Testing Strategy

- Unit tests per evaluation metric (accuracy scorer, hallucination scorer, tier classifier) with fixed synthetic inputs and expected outputs.
- Integration tests for a full mini benchmark run against a mocked provider adapter.
- Contract tests for each provider adapter against the common interface.
- Manual exploratory testing of the dashboard/report export before each phase demo.
- A final validation run comparing at least 3 free-tier models across all capability categories, used as the report shown in the PRD success metrics.

# 7. Documentation Plan

This SDP is one of five baseline documents (URS, SRS, SDP, SAD, PRD) produced before implementation begins, per the requested process. Each subsequent phase updates the SAD if architectural decisions change, and the PRD if scope/priorities change; the SRS is the frozen contract for functional scope for this iteration.